UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
ESCUELA DE POSGRADO
MAESTRÍA EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL



PROYECTO FINAL

Estructuras de datos avanzadas orquestadas para un pipeline eficiente de procesamiento de data streams

Escenario B: sistema de comercio electrónico

GRUPO
Grupo 3

AUTOR(ES)
Javier Guerra Blanco,
Armando Mesias Araujo,
Brayan Quispe Montoya,
Raúl Pacheco Huanca

DOCENTE
Heider Sánchez

CURSO
Estructuras de Datos y Algoritmos — 2026-2

Lima – Perú
2026

Contenido

Resumen 3
1. Contexto y motivación 3
1.1. Situación 3
1.2. Problemas de negocio 3
1.3. Por qué las soluciones exactas no alcanzan 3
1.4. Pregunta y objetivo 4
1.5. Dataset base y componente sintético 4
2. Supuestos 4
3. Arquitectura del pipeline 5
4. Qué hace cada etapa y por qué se eligió su estructura 6
5. Investigación avanzada (etapas 6–8) 8
5.1. Filtrado probabilístico: Bloom Filter y Count-Min Sketch 8
5.2. Similitud de conjuntos: MinHash + LSH 9
5.3. Búsqueda aproximada de vecinos (ANN): HNSW 9
5.4. Ranking de influencia: PageRank y Personalized PageRank 10
5.5. Detección de comunidades: Louvain frente a Label Propagation 10
6. Diseño experimental 10
7. Resultados 11
7.1. Filtrado probabilístico frente a estructuras exactas 11
7.2. Similitud: MinHash+LSH y HNSW frente a métodos exactos 11
7.3. Grafo de co-compra: ranking y comunidades 12
7.4. Pipeline de punta a punta: alertas, limpieza y micro-lotes 12
7.5. Escalabilidad de las estructuras base 13
7.6. Demostración de punta a punta 13
8. Conclusiones 14
9. Referencias 14

Resumen

Este proyecto diseña, implementa y evalúa un pipeline de procesamiento de data streams para una tienda de comercio electrónico multicategoría. El stream se construye a partir de un subconjunto de 10 millones de eventos reales (vistas, carrito y compras) del dataset **eCommerce behavior data from multi-category store** de REES46, al que se añaden campos sintéticos para tipo de orden, medio de pago, dispositivo e inyección de fraude. Cada una de las diez etapas usa la estructura de datos más adecuada a su función: cola FIFO, ordenación y búsqueda binaria, heap, tabla hash con caché LRU, B+Tree, KD-Tree, Bloom Filter con Count-Min Sketch, MinHash+LSH con HNSW, y PageRank con detección de comunidades sobre el grafo de co-compra. La salida de cada etapa alimenta a la siguiente: las etapas ligeras se actualizan con cada evento y las costosas por micro-lotes, por lo que las recomendaciones cambian mientras llega el stream. El sistema termina en un servicio con autocompletado, rankings Top-K, recomendaciones y alertas de pago y de fraude. Los componentes sintéticos y las decisiones de operación se declaran como supuestos explícitos. Las técnicas aproximadas se comparan contra alternativas exactas en tiempo, memoria y calidad, con tamaños de hasta 10M elementos (100K en las técnicas de similitud).

Palabras clave: data streams, Bloom Filter, Count-Min Sketch, MinHash, LSH, HNSW, PageRank, Louvain, comercio electrónico.

1. Contexto y motivación

1.1. Situación

Una tienda en línea multicategoría (electrónica, electrodomésticos, computadoras, ropa) recibe un flujo continuo de eventos: cada vista, cada producto agregado al carrito y cada compra. Solo el dataset público de REES46, que registra la actividad de una tienda de este tipo, reúne cerca de 285 millones de eventos en pocos meses. Cada evento es a la vez una oportunidad de venta y una posible señal de riesgo, y muchas decisiones deben tomarse en el momento: qué atender primero, si una operación es sospechosa y qué recomendar mientras el cliente sigue navegando.

1.2. Problemas de negocio

Problema	Ejemplo concreto	Etapas
Atender primero lo urgente	Una orden express o un pago rechazado no puede esperar detrás de miles de vistas de productos	0–2
Detectar fraude sin frenar a clientes legítimos	Prueba de tarjetas robadas (muchos intentos de pago en minutos) y bots que acaparan productos en el carrito	6
Consultar productos con baja latencia	Ficha, precio, disponibilidad y productos de una categoría dentro de un rango de precio	3–4
Recomendar mientras el cliente navega	"Quienes vieron esto también vieron…" y "se compra junto con…", actualizados con el stream	5, 7, 9
Identificar productos que impulsan la venta cruzada	Un producto puede venderse poco y aun así arrastrar muchas otras compras	8

1.3. Por qué las soluciones exactas no alcanzan

Las estructuras exactas crecen con los datos. Comparar todos los pares de productos para encontrar similares crece como n^2: con 100 mil productos son unos 5 mil millones de comparaciones. Guardar el conteo exacto de eventos de cada usuario en cada ventana de tiempo hace crecer la memoria sin límite, y consultar una lista negra exacta para cada uno de los 10 millones de eventos exige mantenerla completa en memoria. Las estructuras probabilísticas y aproximadas cambian un error pequeño y controlado por memoria fija y respuestas en milisegundos. El reto del proyecto es elegir, en cada etapa, dónde se puede tolerar ese error y dónde la decisión exige verificación exacta. La

recomendación por co-compra tiene además un antecedente industrial conocido: el filtrado colaborativo ítem a ítem de Amazon (Linden et al., 2003).

1.4. Pregunta y objetivo

Pregunta: ¿es posible procesar un stream de 10 millones de eventos de comercio electrónico priorizando órdenes, detectando fraude y generando recomendaciones con memoria acotada y baja latencia, manteniendo un error medible y aceptable frente a las soluciones exactas?

Objetivo: diseñar, implementar y evaluar un pipeline de diez etapas en el que cada etapa use la estructura de datos más adecuada y su salida sea entrada real de las siguientes, y comparar experimentalmente cada técnica aproximada con su alternativa exacta.

1.5. Dataset base y componente sintético

Se usa el dataset de REES46 (Kechinov, 2019) como base real: catálogo de productos e historial de vistas, carritos y compras con timestamps reales, usuario y sesión. Se toman 10 millones de eventos consecutivos para conservar la estructura de las sesiones. Lo que el dataset no trae y el pipeline necesita (tipo de orden, medios de pago y dispositivos, fraude, estado del pago, búsquedas, stock, duplicados, registros inválidos y eventos tardíos) se genera de forma sintética con semilla 42, según los supuestos S2 a S9.

2. Supuestos

Los supuestos siguientes definen cómo se completa el dataset y cómo opera el pipeline. Todos son parámetros del código (`src/preparar_datos.py` y `src/pipeline.py`) y pueden ajustarse.

#	Supuesto	Justificación
S1	Una orden es el conjunto de compras de una misma `user_session`.	El dataset no incluye identificador de orden.
S2	Los timestamps son reales y el stream se reproduce por orden de llegada. ~1% de los eventos llega con 1 a 30 minutos de retraso y se procesa igual, con una tolerancia de 2 horas en las ventanas de conteo.	En un stream real los eventos pueden llegar tarde o desordenados.
S3	Tipo de orden de cada compra: express 10%, estándar 70%, programada 20%.	El dataset no distingue urgencia; el escenario la exige.
S4	Cada usuario tiene de 1 a 3 medios de pago y de 1 a 2 dispositivos.	Permite señalar entidades distintas del usuario.
S5	El 0.5% de los usuarios son defraudadores, con ráfagas de 20 carritos y 6 compras en 2 minutos. Solo la mitad está en la lista negra.	Evalúa por separado la detección por lista (Bloom Filter) y por intensidad (Count-Min Sketch).
S6	Se rechaza el 2% de los pagos normales, el 50% de los de defraudadores y el 60% de los de las ráfagas. Tres rechazos del mismo medio de pago en 10 minutos indican prueba de tarjetas.	Genera alertas de pago, que el enunciado pide priorizar.
S7	Ruido: ~1% de duplicados exactos y ~0.5% de registros inválidos (precio negativo, tipo de evento mal escrito o sesión vacía).	Los datos de origen vienen limpios; sin ruido no se puede evaluar la etapa 1.
S8	El 15% de las sesiones empieza con una búsqueda: la marca o el tipo del primer producto visto, 1 a 30 segundos antes de verlo.	El dataset no registra búsquedas y el autocompletado las necesita.
S9	Stock inicial sintético por producto (5% agotado), sin reposición durante el periodo. Una venta sin stock se registra como pedido pendiente.	El dataset no incluye disponibilidad.
S10	Si el grafo de co-compra tiene menos de 1,000 aristas, se suman los carritos de una misma sesión con peso 0.5.	Las compras con dos o más productos son escasas.
S11	Las etapas 0–4 y el Trie se actualizan evento a evento; las etapas 5, 7 y 8 se refrescan por micro-lotes (por defecto, cada n/4 eventos con un máximo de 1 millón). Las comunidades se calculan al cerrar el stream.	Reconstruir HNSW o el grafo en cada evento sería inviable.
S12	El B+Tree reside en memoria; cada nodo simula una página de disco de orden 64 y las eliminaciones son perezosas.	Simplifica la implementación sin cambiar su complejidad.
S13	La verificación exacta de una alerta simula la consulta a la base de datos transaccional.	Ninguna operación se bloquea solo por una estructura probabilística.

#	Supuesto	Justificación
S14	Umbrales por ventana de 10 minutos: 18 carritos y 6 compras por usuario, y 3 pagos rechazados por medio de pago. Se fijaron por encima del percentil 99.9 observado en los 10M eventos (17 carritos y 5 compras).	Separan el comportamiento normal del anómalo.
S15	Devoluciones y reseñas quedan fuera del alcance.	Ninguna etapa obligatoria las requiere.
S16	La interacción usuario-producto pesa 1 (vista), 3 (carrito) y 5 (compra).	Refleja mayor intención de compra; los pesos no se calibraron.
S17	HNSW indexa los 5,000 productos con más interacción en cada micro-lote.	HNSW está escrito en Python puro y reconstruirlo con todo el catálogo en cada refresco tomaría demasiado tiempo.
S18	En el grafo se ignoran las órdenes con más de 50 productos; en MinHash+LSH solo se indexan productos vistos en al menos 5 sesiones.	Una orden de k productos genera k(k−1)/2 aristas; con pocas sesiones la firma no es representativa.

Parámetros de las estructuras. Se declaran todos los valores usados y de dónde salen. Los que dicen "valor inicial" no se optimizaron.

Parámetro	Valor	Origen o justificación
Tamaño de lote / capacidad de la cola	10,000 / 50,000 eventos	Valor inicial.
Caché LRU	10,000 fichas	Valor inicial; la tasa de aciertos se reporta en 7.4.
Orden del B+Tree	64 claves por nodo	Aproxima una página de disco de 4 KB (S12).
Bloom Filter	p = 1%; m y k según las fórmulas de 5.1	Cada positivo se verifica (S13); una p menor exige más memoria (7.1).
Count-Min Sketch	ε = 0.001, δ = 0.01; ventana 10 min en bloques de 1 min; tolerancia 2 h	Error ≤ 0.1% del total con probabilidad 99% (5.1); la ventana coincide con los umbrales de S14.
MinHash + LSH	256 permutaciones; 128 bandas × 2 filas	Única configuración evaluada que cumple recall ≥ 0.8 para Jaccard ≥ 0.125 (7.2).
SVD + HNSW	32 dimensiones; M = 16; efConstruction = 100	M = 16 es el valor por defecto de hnswlib; efConstruction se bajó de 200 (valor por defecto) a 100 por el tiempo en Python puro.
PageRank	d = 0.85; tolerancia 10^{-6}	d = 0.85 es el valor del artículo original (Brin y Page, 1998).
Comunidades	10 semillas para la estabilidad	Permite promediar la NMI entre 45 pares de ejecuciones.
KD-Tree	hasta 16 puntos por hoja	Valor inicial.

3. Arquitectura del pipeline

La Figura 1 muestra el recorrido de un evento. Las etapas 0, 1, 2, 6, 3, 4 y el Trie de la etapa 9 se ejecutan para cada evento, en ese orden. Las etapas 5, 7 y 8 son más costosas y se refrescan por micro-lotes con lo acumulado en la etapa 3 (S11). Las alertas que produce la etapa 6 vuelven al heap con prioridad máxima. La sección 4 describe qué hace cada etapa, para qué sirve su estructura y por qué se eligió.

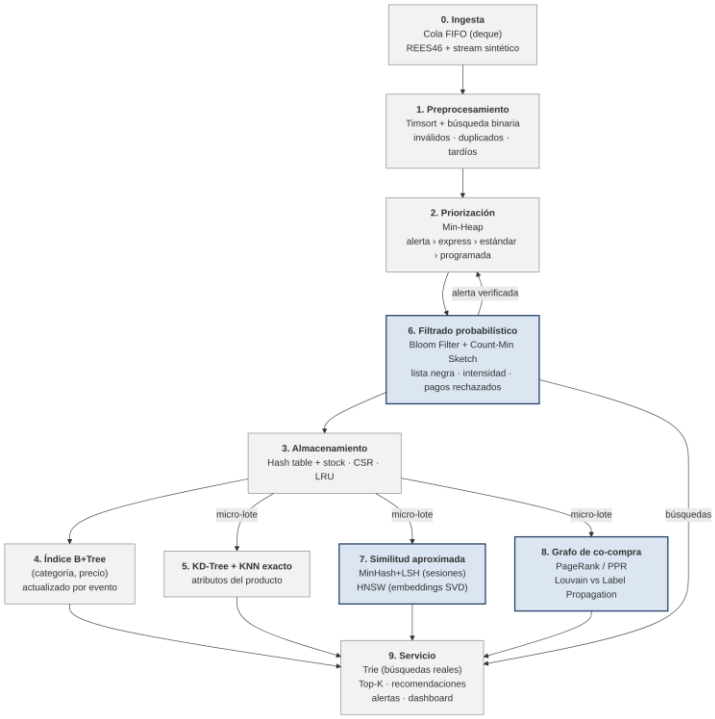


Figura 1. Arquitectura del pipeline de diez etapas. En azul, las etapas fijas [FIJA].

4. Qué hace cada etapa y por qué se eligió su estructura

Cada etapa se describe con tres elementos: **qué hace** con el evento, **para qué sirve** la estructura y **por qué se eligió** frente a la alternativa más directa. Las etapas 6, 7 y 8 se fundamentan en detalle en la sección 5.

Etapa 0 · Ingesta — cola FIFO acotada (por evento)

Qué hace: recibe los eventos en orden de llegada (`arrival_time`, S2) y los entrega en lotes de 10,000 a la etapa 1. Si la cola alcanza su capacidad (50,000), se procesa un lote antes de aceptar más eventos.

Para qué sirve la estructura: separar la llegada de eventos de su procesamiento y limitar cuántos eventos esperan en memoria.

Por qué se eligió: encolar y desencolar cuestan O(1) en `collections.deque`; una lista de Python desencola por el inicio en O(n).

Etapa 1 · Preprocesamiento — ordenación y búsqueda binaria (por lote)

Qué hace: (1) descarta los eventos inválidos y registra el motivo: sesión vacía, tipo de evento desconocido, precio no positivo, producto vacío o búsqueda vacía; (2) normaliza textos (minúsculas, sin espacios; "sin_categoria" y "sin_marca" cuando faltan); (3) ordena el lote por (sesión, producto, tipo, timestamp) y elimina duplicados consecutivos; (4) busca cada clave en las claves del lote anterior para eliminar duplicados que cruzan lotes; (5) cuenta como tardío el evento cuyo timestamp es menor que el máximo ya procesado y lo deja pasar (S2).

Para qué sirve la estructura: que las etapas siguientes reciban solo eventos válidos, únicos y con formato uniforme.

Por qué se eligió: al ordenar (O(b log b)) los duplicados quedan juntos y las claves del lote anterior quedan ordenadas, lo que permite la búsqueda binaria (O(log b)) que pide el enunciado para esta etapa.

Etapa 2 · Priorización — min-heap binario (por evento)

Qué hace: ordena los eventos por (nivel, timestamp, secuencia) y los entrega del nivel menor al mayor: 0 alertas verificadas, 1 compra express, 2 estándar, 3 programada, 4 carrito o búsqueda, 5 vista. Las alertas que genera la etapa 6 vuelven a este heap con nivel 0.

Para qué sirve la estructura: atender primero lo urgente sin ordenar todo el lote cada vez que llega un elemento.

Por qué se eligió: insertar y extraer el mínimo cuesta O(log n); en una lista ordenada insertar cuesta O(n). La secuencia desempata y conserva el orden de llegada dentro de cada nivel.

Etapa 6 · Filtrado probabilístico — Bloom Filter y Count-Min Sketch (por evento) [FIJA]

Qué hace: para cada evento: (1) consulta el Bloom Filter con el usuario, el medio de pago y el dispositivo; un positivo se verifica en la lista exacta y, si se confirma, el evento se bloquea con alerta de lista negra; (2) una compra con pago rechazado genera alerta de pago y suma un rechazo al medio de pago; 3 rechazos en 10 minutos generan alerta de prueba de tarjetas; (3) en carritos y compras, el Count-Min Sketch estima cuántos eventos lleva el usuario en los últimos 10 minutos y, si supera el umbral (S14), se verifica con el conteo exacto antes de alertar. Lo aprobado pasa a la etapa 3; las búsquedas aprobadas, al Trie.

Para qué sirve la estructura: consultar la lista negra y estimar frecuencias con memoria fija, sin importar cuántos eventos lleguen.

Por qué se eligió: un conjunto y un diccionario exactos crecen con los datos. Como ninguna decisión se toma solo con la estructura probabilística (S13), un falso positivo cuesta una consulta exacta adicional, no un bloqueo indebido (sección 5.1).

Etapa 3 · Almacenamiento — tabla hash, caché LRU y matriz dispersa CSR (por evento)

Qué hace: actualiza la ficha del producto (categoría, marca, precio, stock y contadores); descuenta stock en cada compra aprobada (S9); registra la interacción usuario-producto con peso 1 para vista, 3 para carrito y 5 para compra (S16); guarda las sesiones de cada producto (para la etapa 7) y las órdenes (para la etapa 8). Cada vista consulta la ficha a través de la caché LRU de 10,000 fichas.

Para qué sirve la estructura: la tabla hash da acceso a la ficha por su identificador; la caché LRU mantiene en memoria las fichas consultadas recientemente; la matriz CSR guarda solo las celdas con interacción.

Por qué se eligió: la tabla hash consulta en O(1) promedio y la LRU en O(1). Una matriz densa de usuarios × productos reservaría memoria para todas las celdas aunque casi todas estén vacías (la densidad real se reporta en 7.4). La utilidad de la caché depende de que pocos productos concentren las vistas; su tasa de aciertos se mide en 7.4.

Etapa 4 · Índice principal — B+Tree (por evento)

Qué hace: indexa cada producto por la clave (categoría, precio, producto). Inserta los productos nuevos y, si cambia el precio, elimina la clave antigua e inserta la nueva. Responde "productos de la categoría X con precio entre A y B" y devuelve solo los que tienen stock.

Para qué sirve la estructura: resolver consultas puntuales y por rango leyendo pocas páginas.

Por qué se eligió: una tabla hash no permite rangos. Un arreglo ordenado permite rangos con búsqueda binaria, pero cada inserción desplaza elementos (O(n)). El B+Tree busca e inserta en O(log_m n) y recorre el rango por sus hojas enlazadas en O(log_m n + k). El orden 64 simula una página de disco de unos 4 KB (S12).

Etapa 5 · Índice multidimensional — KD-Tree y KNN exacto (micro-lote)

Qué hace: representa cada producto con 4 atributos estandarizados (log del precio, log de vistas, compras/vistas y carritos/vistas) y devuelve los k productos más cercanos de forma exacta. Es la línea base exacta del pipeline.

Para qué sirve la estructura: buscar vecinos exactos en un espacio de pocas dimensiones.

Por qué se eligió: en baja dimensión el KD-Tree descarta ramas completas y evita comparar con todos los productos. El R-Tree está pensado para rectángulos y datos espaciales, y aquí los productos son puntos. Con muchas dimensiones el KD-Tree pierde esa ventaja; por eso los embeddings de 32 dimensiones usan HNSW (etapa 7) y la comparación se mide en 7.2.

Etapa 7 · Similitud aproximada — MinHash+LSH y HNSW (micro-lote) [FIJA]

Qué hace: MinHash+LSH recalcula la firma solo de los productos con sesiones nuevas y la actualiza en las tablas LSH; responde con productos vistos en las mismas sesiones ("similares"). HNSW calcula embeddings de 32 dimensiones con SVD de la matriz CSR para los 5,000 productos más populares (S17), reconstruye el índice y responde con vecinos por similitud coseno ("complementarios").

Para qué sirve la estructura: encontrar productos parecidos sin comparar todos los pares.

Por qué se eligió: la comparación exhaustiva crece con el cuadrado del número de productos; las dos técnicas evitan la mayoría de las comparaciones a cambio de un error que se mide frente a la fuerza bruta (secciones 5.2, 5.3 y 7.2).

Etapa 8 · Grafo de dominio — PageRank y comunidades (micro-lote; comunidades al cierre) [FIJA]

Qué hace: une con una arista cada par de productos comprados en la misma orden; el peso es el número de órdenes compartidas (S1, S10, S18). Calcula PageRank para medir la influencia de cada producto y, en el servicio, Personalized PageRank desde el producto consultado para recomendar co-compras. Al cerrar el stream detecta comunidades con Louvain y Label Propagation y mide su estabilidad.

Para qué sirve la estructura: identificar productos influyentes y grupos de consumo para la venta cruzada.

Por qué se eligió: el grado solo cuenta conexiones directas; PageRank pondera también la importancia de los vecinos. Las alternativas de comunidades se comparan en 7.3 (secciones 5.4 y 5.5).

Etapa 9 · Servicio — Trie, Top-K y panel (por evento y micro-lote)

Qué hace: inserta cada búsqueda aprobada en el Trie y, en cada micro-lote, los nombres de categorías y marcas ponderados por sus vistas; autocompleta un prefijo con los términos más populares. Mantiene el Top-K de productos más vendidos, combina las recomendaciones de las etapas 5, 7 y 8, responde las consultas por rango de la etapa 4 y muestra métricas y alertas en un panel (Streamlit).

Para qué sirve la estructura: responder al usuario final y mostrar el estado del pipeline.

Por qué se eligió: el Trie encuentra un prefijo en O(L), donde L es su longitud, sin recorrer todos los términos; el Top-K con heap cuesta O(n log K) frente a O(n log n) de ordenar todo.

5. Investigación avanzada (etapas 6–8)

5.1. Filtrado probabilístico: Bloom Filter y Count-Min Sketch

Fundamento. El Bloom Filter (Bloom, 1970) representa un conjunto con un arreglo de m bits y k funciones hash. Insertar pone a 1 las k posiciones; consultar responde "posiblemente presente" si todas están en 1 y "ausente" con certeza si alguna está en 0. La probabilidad de falso positivo es aproximadamente (1 − e^(−kn/m))^k y se minimiza con k = (m/n)·ln 2. El Count-Min Sketch (Cormode y Muthukrishnan, 2005) usa una matriz de d filas por w contadores; cada fila tiene su propia función

hash. Una actualización suma en una celda por fila y la estimación es el mínimo de esas celdas. Nunca subestima y, con w = ⌈e/ε⌉ y d = ⌈ln(1/δ)⌉, sobreestima como máximo ε·N con probabilidad 1 − δ.

Diseño. El Bloom Filter guarda las entidades señaladas (usuarios, medios de pago y dispositivos); n es el tamaño real de la lista negra. Como referencia de dimensionamiento, para n = 100,000 entidades y p = 1% se requieren m ≈ 958,506 bits (~117 KB) y k = 7. El Count-Min Sketch estima dos intensidades: eventos por `user_id|event_type` (carrito y compra) y pagos rechazados por medio de pago (prueba de tarjetas). Usa ε = 0.001 y δ = 0.01 (w = 2,719 y d = 5, ~54 KB por sketch). La ventana es deslizante de 10 minutos, formada por un sketch por bloque de 1 minuto; se conservan 2 horas de bloques para aceptar eventos tardíos (S2). Los umbrales son los del supuesto S14. Ninguna operación se bloquea solo por la estructura probabilística: un positivo del Bloom Filter se verifica contra la lista exacta y una alerta del sketch se confirma con un contador exacto para esa clave.

Complejidad y trade-offs. Ambas estructuras insertan y consultan en O(k) u O(d), con memoria fija independiente del número de eventos. El precio es el error: falsos positivos en el Bloom Filter (sin falsos negativos) y sobreestimación en el sketch. El Bloom Filter clásico no admite borrados; si una entidad deja de estar señalada se reconstruye el filtro o se usa un Counting Bloom Filter. **Referencias de implementación:** `pybloom-live`, RedisBloom. **Comparación obligatoria:** set y diccionario exactos de Python en tiempo, memoria, tasa de falsos positivos y error de frecuencia.

5.2. Similitud de conjuntos: MinHash + LSH

Fundamento. MinHash (Broder, 1997) se basa en que, para una permutación aleatoria, la probabilidad de que dos conjuntos tengan el mismo mínimo es igual a su similitud de Jaccard. Con k funciones hash se obtiene una firma de k valores cuyo error estándar es del orden de 1/√k. LSH (Indyk y Motwani, 1998) divide la firma en b bandas de r filas y agrupa en un mismo bucket los conjuntos que coinciden en alguna banda. Dos conjuntos con similitud s son candidatos con probabilidad 1 − (1 − s^r)^b, una curva en S con umbral aproximado (1/b)^(1/r).

Diseño. Cada producto se representa por el conjunto de sesiones en las que fue visto o agregado al carrito. Así, dos productos son similares si los mismos clientes los consideran juntos, lo que sirve para "productos similares". Solo se indexan productos con al menos 5 sesiones (S18). Se consideran similares los pares con Jaccard ≥ 0.125, porque la similitud de co-vista entre productos es baja. Con datos reales se compararon cuatro configuraciones (7.2): con k = 128 y 64 bandas el recall fue 0.77, por debajo del criterio de 0.8; con k = 256 funciones y b = 128 bandas de r = 2 filas (umbral ≈ 0.088) fue 0.95, y esa es la configuración usada. Se eligió MinHash en lugar de SimHash porque los datos son conjuntos y la medida natural es Jaccard; SimHash aproxima la similitud coseno de vectores y es más útil para detectar casi duplicados de texto.

Complejidad y trade-offs. La firma cuesta O(k·|S|) por producto y la consulta es sublineal en el número de productos, frente a O(n²·|S|) de comparar todos los pares. El riesgo son los falsos negativos (pares similares que nunca comparten banda) y los falsos positivos, que se depuran calculando Jaccard exacto sobre los candidatos. **Referencias de implementación:** `datasketch` (MinHashLSH). **Comparación obligatoria:** Jaccard exacto por fuerza bruta en tiempo, memoria y calidad (precisión y recall de los pares sobre el umbral).

5.3. Búsqueda aproximada de vecinos (ANN): HNSW

Fundamento. HNSW (Malkov y Yashunin, 2020) construye un grafo de proximidad navegable organizado en capas: las capas superiores tienen pocos nodos y enlaces largos, y la capa base contiene todos los puntos. La búsqueda desciende de forma voraz desde la capa superior y refina en la base con una lista de candidatos de tamaño efSearch. Su complejidad empírica de búsqueda es cercana a O(log n).

Diseño. Los embeddings de producto se obtienen con SVD truncada (32 dimensiones) de la matriz dispersa usuario × producto de la etapa 3, y se comparan con similitud coseno. Esto permite recomendaciones complementarias basadas en patrones de compra, no solo en co-ocurrencia

directa. Parámetros: M = 16 y efConstruction = 100; se evalúan efSearch ∈ {10, 50, 100} para trazar la curva recall@10 frente a latencia. Se descartó IVF porque requiere entrenar centroides y ajustar nprobe, y M-Tree/VP-Tree porque el espacio es euclidiano. **Referencias de implementación:** `hnswlib`, FAISS (IndexHNSWFlat). **Comparación obligatoria:** KNN exacto por fuerza bruta (producto matricial con NumPy) y KD-Tree, en tiempo, recall@K y memoria. Se espera que el KD-Tree pierda su ventaja en 32 dimensiones por la maldición de la dimensionalidad.

5.4. Ranking de influencia: PageRank y Personalized PageRank

Fundamento. PageRank (Brin y Page, 1998) asigna a cada nodo la probabilidad estacionaria de un paseo aleatorio que, con probabilidad d, sigue una arista (proporcional a su peso) y, con probabilidad 1 − d, salta a un nodo al azar: $PR(v) = (1 - d)/N + d \cdot \Sigma\, PR(u) \cdot w(u,v)/w(u)$. Personalized PageRank reemplaza el salto uniforme por un salto hacia un conjunto semilla.

Diseño. El grafo tiene productos como nodos y una arista ponderada entre cada par de productos comprados en la misma orden; el peso es el número de órdenes compartidas. Se usa d = 0.85 y tolerancia 10^{-6} con iteración de potencias, de costo O(iteraciones·(N + E)). PageRank identifica los productos influyentes para la venta cruzada, y Personalized PageRank, con el producto consultado como semilla, genera recomendaciones de co-compra. **Comparación obligatoria:** ranking por grado ponderado, midiendo la correlación de Spearman entre ambos rankings y comparando sus Top-10.

5.5. Detección de comunidades: Louvain frente a Label Propagation

Fundamento. Louvain (Blondel et al., 2008) maximiza la modularidad $Q = (1/2m) \cdot \Sigma[A_{ij} - k_i \cdot k_j/2m] \cdot \delta(c_i, c_j)$ en dos fases que se repiten: mueve cada nodo a la comunidad vecina que más aumenta Q y luego agrega cada comunidad en un supernodo. Es casi lineal en la práctica, pero depende del orden de visita y puede producir comunidades mal conectadas, defecto que corrige Leiden (Traag et al., 2019). Label Propagation (Raghavan et al., 2007) asigna a cada nodo la etiqueta más frecuente entre sus vecinos hasta converger; es O(E) por iteración y no optimiza ninguna función objetivo.

Diseño. Las comunidades del grafo de co-compra representan agrupaciones de consumo (por ejemplo, accesorios que se compran con un mismo equipo). Se implementa Louvain como algoritmo principal y Label Propagation como alternativa más simple. **Comparación obligatoria:** modularidad, tiempo, número y distribución de tamaños de comunidades, y estabilidad medida con NMI entre 10 ejecuciones con semillas distintas. **Referencias de implementación:** `networkx` (louvain_communities, label_propagation_communities), `python-igraph` y `leidenalg`.

6. Diseño experimental

- **Tamaños:** 1K, 10K, 100K y 1M elementos para las estructuras base y el filtrado probabilístico; hasta 100K para MinHash+LSH y HNSW. El stream de 10M eventos se generó completo y se usó en la preparación y en las comparaciones con datos reales (Paso 5); el benchmark de escalabilidad y el pipeline de punta a punta se ejecutaron hasta 1M por el límite de tiempo de una sesión de Google Colab. La fuerza bruta de Jaccard compara todos los pares hasta 3,000 conjuntos; desde 10K, por su costo cuadrático, la precisión y el recall se estiman con 300 consultas de muestra contra todos los conjuntos.
- **Datos:** los benchmarks de escalabilidad usan datos generados con semilla 42 (claves únicas, frecuencias con distribución Zipf, conjuntos y vectores agrupados). La comparación de configuraciones del notebook de Colab (Paso 5) usa datos reales del stream.
- **Repeticiones:** 100 hasta 10K elementos, 10 para 100K y 3 para 1M, porque una sola construcción a gran escala tarda de segundos a minutos. Se reportan promedio y desviación estándar, y el archivo de resultados registra las repeticiones usadas en cada fila.
- **Plataforma y semilla:** Google Colab, Linux 6.6 x86_64, Python 3.13.18; semilla 42 para todos los componentes sintéticos y aleatorios.

- **Medición:** `time.perf_counter` para tiempos y `tracemalloc` para la memoria pico.
- **Criterios de aceptación:** Bloom Filter con tasa de falsos positivos ≤ 1.5 × p; Count-Min Sketch con error máximo ≤ ε·N; MinHash+LSH con recall ≥ 0.8 para los pares con Jaccard ≥ 0.125; HNSW con recall@10 ≥ 0.9; PageRank aporta un orden distinto al del grado (Spearman < 1); Louvain logra mayor modularidad que Label Propagation con estabilidad comparable.
- **Escalabilidad:** gráficos log-log de tiempo frente a n, contrastando la pendiente con la complejidad esperada (pendiente 1 para O(n), cercana a 1 para O(n log n)).

7. Resultados

Cada tabla responde a una de las comparaciones obligatorias. Las tablas 7.1, 7.2 y 7.5 provienen del benchmark (`experiments/resultados/benchmark.csv`); las comparaciones con datos reales y las tablas 7.3, 7.4 y 7.6 provienen del notebook de Colab.

7.1. Filtrado probabilístico frente a estructuras exactas

n	Bloom: tiempo / memoria	Set exacto: tiempo / memoria	Tasa FP (teórica 1%)	CMS: error promedio	Dict exacto: memoria
1K	11.2 ms / 0.002 MB	0.07 ms / 0.04 MB	1.30%	0.000	0.01 MB
10K	34.7 ms / 0.01 MB	0.31 ms / 0.66 MB	1.07%	0.001	0.06 MB
100K	396 ms / 0.12 MB	6.2 ms / 6.29 MB	0.98%	1.371	0.44 MB
1M	3,523 ms / 1.20 MB	84 ms / 50.33 MB	0.97%	25.637	3.94 MB

El Count-Min Sketch usa 0.06 MB en todos los tamaños; su error máximo a 1M fue 118, dentro de la cota ε·N = 1,000. El Bloom Filter mantiene la tasa de falsos positivos cerca del 1% teórico y, a 1M, ocupa 42 veces menos memoria que el set. Es más lento porque está escrito en Python, mientras que el set de Python está implementado en C.

Con datos reales (Paso 5, stream de 10M). Bloom Filter sobre 200,000 usuarios: con p = 0.01 la tasa medida fue 1.08% con 0.24 MB, frente a 12.58 MB del set (52 veces menos); con p = 0.05 y p = 0.001 las tasas fueron 5.0% y 0.087%, y las tres cumplen el criterio. Count-Min Sketch sobre los 310,400 eventos de carrito: con ε = 0.01 el error máximo fue 1,278 (cota 3,104) usando 0.005 MB; con ε = 0.001, 167 (cota 310) usando 0.054 MB, frente a 3.85 MB del diccionario. Ambos cumplen; el pipeline usa ε = 0.001.

7.2. Similitud: MinHash+LSH y HNSW frente a métodos exactos

n	LSH: tiempo	Fuerza bruta Jaccard: tiempo	LSH: precisión / recall	HNSW: tiempo / recall@10	KNN exacto: tiempo
1K	138 ms	2,111 ms (todos los pares)	0.860 / 0.999	44.7 ms / 1.000	8.9 ms
10K	2,119 ms	7,549 ms (300 consultas)	0.403 / 1.000	48.9 ms / 0.932	56.1 ms
100K	31,764 ms	72,453 ms (300 consultas)	0.064 / 1.000	57.0 ms / 0.862	898.8 ms

Pares similares: Jaccard ≥ 0.125 (criterio de 5.2). La columna LSH: tiempo es la construcción del índice; la consulta de pares tomó 0.13 s a 100K, frente a 72 s de la fuerza bruta. Desde 10K, la fuerza bruta y el recall se calculan con 300 consultas de muestra. HNSW con efSearch = 50; tiempos de HNSW y KNN exacto para 100 consultas de 10 vecinos.

LSH mantiene recall ≈ 1.0 en todos los tamaños. Su precisión de candidatos baja con n (0.064 a 100K) porque aparecen más pares por azar, pero esos candidatos se verifican con Jaccard exacto y no llegan al usuario. El costo es la memoria del índice: 1.7 GB a 100K.

HNSW responde 16 veces más rápido que el KNN exacto a 100K, pero en el benchmark su recall@10 baja con n: 0.932 a 10K y 0.862 a 100K con ef = 50 (0.885 con ef = 100). Los vectores del benchmark son sintéticos, con 1,000 grupos a 100K, lo que hace más difícil la búsqueda. Con los embeddings

reales del Paso 5 (5,000 productos), HNSW sí cumple: recall@10 de 0.966 con ef = 10, 0.997 con ef = 50 y 0.998 con ef = 100.

El KD-Tree en 32 dimensiones devuelve los vecinos exactos (recall 1.0), pero es más lento que la fuerza bruta: 7.05 s frente a 0.90 s a 100K. Es la maldición de la dimensionalidad, y justifica usar HNSW para embeddings. En 2 dimensiones (atributos) el KD-Tree sí rinde: a 1M responde 100 consultas en 36 ms, frente a 3.98 s de la fuerza bruta.

Configuraciones de LSH con datos reales (2,000 productos; 154 pares con Jaccard ≥ 0.125; fuerza bruta: 181 s). k = 128 con 16 bandas: recall 0.00; con 32 bandas: 0.02; con 64 bandas: 0.77 en 0.56 s; k = 256 con 128 bandas: **0.95** en 1.5 s. Solo la última cumple el criterio y es la que usa el pipeline.

7.3. Grafo de co-compra: ranking y comunidades

Métrica	Louvain	Label Propagation
Modularidad Q	0.845	0.778
Tiempo (s)	0.070	0.074
Número de comunidades	480	563
Tamaño mediano / máximo	2 / 182	2 / 68
Estabilidad (NMI promedio, 10 semillas)	0.980	0.975

Grafo de compras: 1,702 productos y 1,991 aristas (grado promedio 2.3). Louvain obtiene mayor modularidad que Label Propagation, en un tiempo similar y con estabilidad comparable, por lo que cumple el criterio. El tamaño mediano de 2 refleja un grafo disperso, con muchos pares de productos aislados.

PageRank frente a grado ponderado: correlación de Spearman 0.322 sobre todos los nodos y 10 productos en común en los Top-10. Los productos más influyentes coinciden con los de mayor grado; la diferencia está en el resto del ranking, donde PageRank favorece a los productos conectados con productos centrales. Cumple el criterio (Spearman < 1), pero el aporte para venta cruzada está en la parte media del ranking, no en el Top-10.

7.4. Pipeline de punta a punta: alertas, limpieza y micro-lotes

Métrica	Resultado
Eventos procesados	1,000,000 (primer millón del stream de 10M)
Registros inválidos descartados (por motivo) / duplicados / tardíos	6,677 (precio inválido 3,409; sesión vacía 1,666; tipo desconocido 1,602) / 10,227 / 8,416
Alertas por motivo: lista negra · intensidad · prueba de tarjetas · pago rechazado	8,327 · 258 · 162 · 957
Precisión de las alertas de fraude frente al fraude inyectado	0.997
Falsos positivos del Bloom Filter descartados en la verificación	30,209 de 38,536 positivos
Tasa de aciertos de la caché LRU / productos agotados	77.7% / 3,463
Densidad de la matriz usuario-producto	0.000060 (0.006%)
Refrescos por micro-lote y cambio de la recomendación del producto seguido	5 refrescos; la recomendación cambió en 3 de 4 transiciones

La verificación exacta (S13) es necesaria: la lista negra es pequeña frente a millones de consultas, así que, aunque la tasa de falsos positivos del Bloom Filter es cercana al 1%, el 78% de sus positivos fueron falsos. Ninguno llegó a bloquear una operación.

7.5. Escalabilidad de las estructuras base

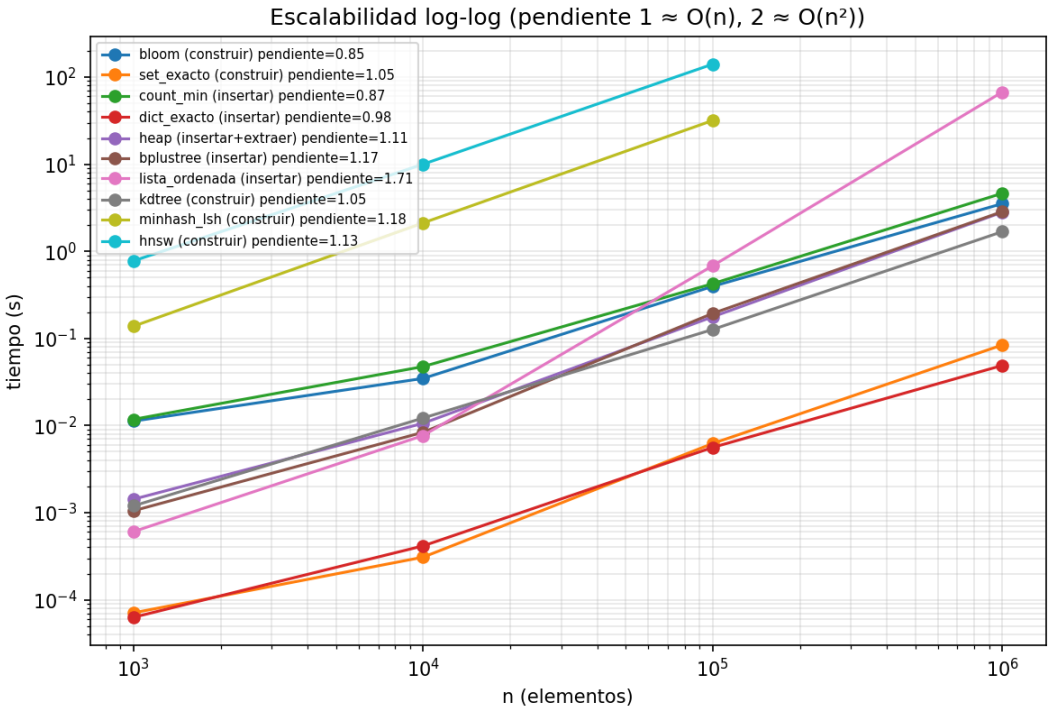


Figura 2. Tiempo de construcción o inserción frente a n (escala log-log), 1K a 1M elementos.

Estructura	Pendiente medida	Complejidad esperada	Lectura
Bloom Filter (construir)	0.85	O(n)	Lineal; menor a 1 por costo fijo en n pequeños
Count-Min Sketch (insertar)	0.87	O(n)	Lineal; memoria constante (0.06 MB)
Heap (insertar + extraer)	1.11	O(n log n)	Coincide
B+Tree (insertar)	1.17	O(n log n)	Coincide; altura 4 a 1M
Lista ordenada (insertar)	1.71	$O(n^2)$	Crece casi cuadrático: 67 s a 1M frente a 2.9 s del B+Tree
KD-Tree (construir)	1.05	O(n log n)	Coincide
MinHash + LSH (construir)	1.18	O(n·k)	Lineal en n con k = 256 fijo
HNSW (construir)	1.13	O(n log n)	Coincide; 141 s a 100K en Python puro

La comparación clave para las consultas por rango es B+Tree frente a lista ordenada: con 1M productos, el B+Tree inserta 24 veces más rápido y responde un rango del 1% en 18 ms.

7.6. Demostración de punta a punta

Traza de una compra express real (producto 1002532) a través de las 10 etapas:

- **Ingesta (0):** el evento entra a la cola FIFO.
- **Preprocesamiento (1):** pasa la validación, la normalización y la deduplicación.
- **Priorización (2):** el heap lo atiende con prioridad 1 (express), antes que las órdenes estándar y las vistas.
- **Filtrado (6):** el Bloom Filter no encuentra entidades señaladas, el pago está aprobado y el Count-Min Sketch está bajo el umbral, así que la operación se aprueba.
- **Almacenamiento (3):** se actualizan la ficha, el stock (quedan 5 unidades), la matriz usuario-producto y las órdenes.
- **Índice por rango (4):** el B+Tree indexa el producto por (categoría, precio = 642.69).

- **Micro-lote (5, 7, 8):** KD-Tree, MinHash+LSH, HNSW y el grafo incorporan el producto en cada refresco.
- **Servicio (9):** recomendaciones Top-5 del producto: similares por comportamiento (LSH) 1003363 y 45600297; complementarios (HNSW) 1002535, 1002528 y 1003363; co-compra (Personalized PageRank) 4804056, 1005115 y 1004856; por atributos (KNN) 1801849, 1002540 y 1306359.

Las ráfagas sintéticas de los usuarios con fraude inyectado (S5) produjeron 258 alertas por intensidad y 162 por prueba de tarjetas, todas verificadas antes de bloquear; la precisión frente al fraude inyectado fue 0.997.

8. Conclusiones

- **Filtrado probabilístico:** el Bloom Filter y el Count-Min Sketch cumplen en datos sintéticos y reales. La tasa de falsos positivos quedó entre 0.97% y 1.30% (teórica 1%) y el error del Count-Min siempre bajo ε·N, con 42 y 70 veces menos memoria que el set y el diccionario a 1M. Como el 78% de los positivos del Bloom Filter en el pipeline fueron falsos, la verificación exacta antes de bloquear es indispensable.
- **Similitud:** MinHash+LSH cumple solo con 256 permutaciones y 128 bandas (recall 0.95 con datos reales), porque la similitud de co-vista es baja; el costo es más memoria y más candidatos falsos, que se descartan con Jaccard exacto. HNSW cumple con embeddings reales (recall@10 ≥ 0.966) y es 16 veces más rápido que el KNN exacto a 100K, pero con 100K vectores sintéticos su recall baja a 0.86–0.89. El KD-Tree no sirve en 32 dimensiones y sí en atributos de baja dimensión.
- **Grafo:** Louvain supera a Label Propagation en modularidad (0.845 frente a 0.778) con tiempo y estabilidad similares. PageRank coincide con el grado en el Top-10 y difiere en el resto del ranking (Spearman 0.32).
- **Escalabilidad:** las pendientes medidas coinciden con la complejidad esperada (0.85–1.18). La lista ordenada (pendiente 1.71) confirma por qué se usa un B+Tree para las consultas por rango.
- **Límites y mejoras:** el grafo de compras es disperso (grado promedio 2.3), lo que da comunidades pequeñas; se propone agregar co-carrito con menor peso. HNSW necesita un efSearch o un M mayor para mantener recall ≥ 0.9 con n grande. El pipeline y el benchmark se ejecutaron hasta 1M por el tiempo de Colab; implementar las estructuras en C o con NumPy (o usar `faiss`/`datasketch`) permitiría procesar los 10M completos.

9. Referencias

Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. Journal of Statistical Mechanics: Theory and Experiment, 2008(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. Communications of the ACM, 13(7), 422–426. https://doi.org/10.1145/362686.362692

Brin, S., & Page, L. (1998). The anatomy of a large-scale hypertextual Web search engine. Computer Networks and ISDN Systems, 30(1–7), 107–117. https://doi.org/10.1016/S0169-7552(98)00110-X

Broder, A. Z. (1997). On the resemblance and containment of documents. In Proceedings of the Compression and Complexity of Sequences (pp. 21–29). IEEE. https://doi.org/10.1109/SEQUEN.1997.666900

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Cormode, G., & Muthukrishnan, S. (2005). An improved data stream summary: The Count-Min sketch and its applications. Journal of Algorithms, 55(1), 58–75. https://doi.org/10.1016/j.jalgor.2003.12.001

Indyk, P., & Motwani, R. (1998). Approximate nearest neighbors: Toward removing the curse of dimensionality. In Proceedings of the 30th Annual ACM Symposium on Theory of Computing (pp. 604–613). ACM. https://doi.org/10.1145/276698.276876

Kechinov, M. (2019). eCommerce behavior data from multi category store [Dataset]. Kaggle / REES46 Open CDP. https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store

Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). Mining of massive datasets (3rd ed.). Cambridge University Press.

Linden, G., Smith, B., & York, J. (2003). Amazon.com recommendations: Item-to-item collaborative filtering. IEEE Internet Computing, 7(1), 76–80. https://doi.org/10.1109/MIC.2003.1167344

Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

Raghavan, U. N., Albert, R., & Kumara, S. (2007). Near linear time algorithm to detect community structures in large-scale networks. Physical Review E, 76(3), 036106. https://doi.org/10.1103/PhysRevE.76.036106

Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden: Guaranteeing well-connected communities. Scientific Reports, 9, 5233. https://doi.org/10.1038/s41598-019-41695-z